Chapter 2: Linked Lists

Technical / interview questions, exercises, multiple choice questions and answer key for Chapter 2 (Linked Lists) of Data Structures.

Programming Exercises

1. Implement a basic singly linked list in C.
2. Write a function to insert a node at the beginning of a singly linked list.
3. Develop a program to insert a node at the end of a singly linked list.
4. Create a function to delete a specific node from a singly linked list.
5. Write a program to reverse a singly linked list.
6. Implement a function to find the length of a linked list.
7. Write a program to search for an element in a linked list.
8. Develop a function to concatenate two linked lists.
9. Implement a program to copy a linked list.
10. Write a function to detect a loop in a linked list.
11. Implement a basic doubly linked list in C.
12. Write a function to insert a node at the beginning of a doubly linked list.
13. Develop a program to insert a node at the end of a doubly linked list.
14. Create a function to delete a specific node from a doubly linked list.
15. Write a program to reverse a doubly linked list.
16. Implement a basic circular linked list in C.
17. Write a function to insert a node at the beginning of a circular linked list.
18. Develop a program to insert a node at the end of a circular linked list.
19. Create a function to delete a specific node from a circular linked list.
20. Write a program to reverse a circular linked list.
21. Implement a linear search in an array.
22. Implement a linear search in a linked list.
23. Compare the time complexity of searching in an array vs. a linked list.
24. Implement a simple array-based data structure and compare it with a linked list.
25. Implement a stack using a linked list.
26. Implement a queue using a linked list.
27. Write a program to represent a polynomial using a linked list.
28. Develop a program to simulate a simple file system using linked lists.
29. Implement a basic memory allocator using linked lists.

Technical / Interview Questions

1. What is a linked list?
2. Explain the basic concept of a node in a linked list.
3. Differentiate between an array and a linked list.
4. Why are linked lists considered dynamic data structures?
5. How is a singly linked list different from a doubly linked list?
6. Write a function to insert a node at the beginning of a singly linked list.
7. Explain the concept of a tail pointer in a singly linked list.

8. Describe the process of reversing a singly linked list.
9. What is the time complexity of inserting an element at the end of a singly linked list?
10. How do you find the length of a linked list?
11. Write a program to search for a specific element in a linked list.
12. Explain the difference between a shallow copy and a deep copy of a linked list.
13. Describe the procedure for concatenating two linked lists.
14. How can you detect a loop in a linked list?
15. What are the advantages of a doubly linked list over a singly linked list?
16. Write a function to delete a node from the middle of a doubly linked list.
17. Explain the process of reversing a doubly linked list.
18. How does a doubly linked list facilitate bidirectional traversal?
19. What is a circular linked list, and where is it useful?
20. Write a program to insert a node at the end of a circular linked list.
21. How do you detect the presence of a loop in a circular linked list?
22. Explain the advantages of using a circular linked list in certain scenar ios.
23. Compare the time complexity of searching in an array vs. a linked list.
24. Discuss the memory utilization differences between arrays and linked lists.
25. In what scenarios would you prefer using an array over a linked list, and vice versa?
26. How can you implement a stack using a linked list?
27. Explain the application of linked lists in representing polynomials.
28. Describe a real-world scenario where using a linked list is advanta geous.
29. How would you implement a queue using a linked list?
30. Discuss the role of linked lists in a file system.

Multiple Choice Questions

1. What is a linked list?

A. A linear data structure
B. A two-dimensional array
C. A stack
D. A tree

2. Which of the following is a key feature of linked lists?

A. Fixed size
B. Contiguous memory allocation
C. Dynamic size
D. Random access

3. In a linked list, what is a node?

A. A connected link between elements
B. The first element
C. The last element
D. A data structure for sorting

4. Why are linked lists considered dynamic data structures?

A. They use dynamic memory allocation
B. They have a fixed size
C. They use contiguous memory allocation
D. They allow random access

5. What is the main disadvantage of singly linked lists?

A. They require more memory
B. They cannot be traversed in both directions
C. They have slower access time
D. They cannot store large datasets

6. How is a singly linked list different from a doubly linked list?

A. Singly linked lists allow bidirectional traversal
B. Singly linked lists have two pointers per node
C. Singly linked lists only have a forward link
D. Singly linked lists use contiguous memory

7. What is the time complexity of searching for an element in a singly linked list?

A. O(1)
B. O(log n)
C. O(n)
D. $O(n^2)$

8. How do you find the length of a linked list?

A. Count the number of nodes
B. Use the sizeof() function
C. Check the head pointer
D. Traverse the list twice

9. What is a common operation on linked lists for searching an element?

A. Linear search
B. Binary search
C. Hashing
D. Sorting

10. What is the purpose of the tail pointer in a linked list?

A. To point to the previous node
B. To indicate the end of the list
C. To store the data value
D. To point to the first node

11. What is a key advantage of doubly linked lists over singly linked lists?

A. Lower memory usage
B. Simpler implementation
C. Bidirectional traversal
D. Faster access time

12. How is deleting a node from a doubly linked list different from delet ing a node in a singly linked list?

A. It requires more memory
B. It involves updating more pointers
C. It has a higher time complexity
D. It cannot be done in constant time

13. What is a circular linked list?

A. A list with circular elements
B. A list that is always sorted
C. A list where the last node points to the first node
D. A list without a tail pointer

14. What is the primary advantage of using a circular linked list?

A. Faster insertion at the beginning
B. Constant time for reversing the list
C. It prevents memory leaks
D. Bidirectional traversal

15. Which of the following statements is true regarding memory alloca tion in arrays and linked lists?

A. Arrays use dynamic memory allocation
B. Linked lists use contiguous memory allocation
C. Arrays have a fixed size
D. Linked lists allow random access

16. In terms of searching, what is a disadvantage of using linked lists compared to arrays?

A. Slower access time
B. Lower memory usage
C. Constant time complexity
D. Easier implementation

17. How can you implement a stack using a linked list?

A. Use a circular linked list
B. Use a doubly linked list
C. Use a singly linked list
D. Use an array

18. Explain the application of linked lists in representing polynomials.

A. Linked lists simplify polynomial evaluation
B. Linked lists help in polynomial division
C. Linked lists provide a compact representation for polynomials
D. Linked lists prevent polynomial multiplication

19. In what scenario is using a linked list advantageous over an array?

A. When random access is required
B. When a fixed-size structure is needed
C. When dynamic memory allocation is essential
D. When a compact memory layout is preferred

Answer Key

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
1	A	2	C	3	A	4	A	5	B
6	C	7	C	8	A	9	A	10	B
11	C	12	B	13	C	14	A	15	C
16	A	17	C	18	C	19	C		